Tutorial Problem Set #3

- Sample executables can be found in the course git repository directory (run `git pull`).
- Completing the problem set will reduce the weight of the final exam by 0.5%. To complete a problem set, you must pass at least 50% of the secret tests.
- You may assume all input is valid. Tutorial problems NEVER require checking for invalid inputs.
- You may only import the following libraries: `compare`, `iostream`, and `string`.

Question

In the card game *Big Two*, cards are compared in a way that differs from most other games:

- The ranks, from lowest to highest, are `3 4 5 6 7 8 9 T J Q K A 2`. Note that the **2 is the highest rank** and the 3 is the lowest. (`T` stands for ten.)
- The suits, from lowest to highest, are `D C H S` (Diamonds, Clubs, Hearts, Spades). This is *not* alphabetical order.
- A card with a higher rank always beats a card with a lower rank, whatever the suits. Only when two cards have the same rank does the suit decide. For example, `2D` beats `AS`, and `7S` beats `7H`.

A card is represented by the struct below, together with two strings that list the ranks and suits in increasing order:

```
const string RANKS = "3456789TJQKA2";
const string SUITS = "DCHS";

struct Card {
  char rank;  // one of the characters in RANKS
  char suit;  // one of the characters in SUITS
  // ...
};
```

Reading a card from a stream and printing a card to a stream are already done for you. A card is written as its rank followed by its suit, with no space, for example `7H` or `AS`.

You will implement the three-way comparison operator `operator<=>` as a member of `Card`, so that cards are ordered as described above.

Hints. Comparing the characters directly will *not* work: for example, `'K' < 'T'` in ASCII, but a king is higher than a ten, and `'C' < 'D'` even though diamonds are lower than clubs. Since `RANKS` and `SUITS` list the characters in increasing order, the position of a character in the string tells you how high it is. The member function `find` of `string` returns that position: for example, `RANKS.find('4')` is `1`.

Test harness

Starter files are provided: you are given a sample executable, starter code, and a sample input/output. You are given a test harness that tests the operator you implement. The harness keeps a hand of up to 13 cards and supports the following commands:

- `q`: Quits the program.
- `d 7H`: Deals a card, adding `7H` to the end of the hand. `7H` can be replaced with any valid card.
- `p`: Prints the hand on one line, separated by spaces.
- `s`: Sorts the hand from lowest to highest. The sort uses only `operator<`, which the compiler rewrites in terms of your `operator<=>`.

Sample

Input:

```
d 3D
d 2S
d KH
p
s
p
q
```

Output:

```
3D 2S KH
3D KH 2S
```

Submission

Submit your solution in a file called `main.cc`. Save your file in the folder `tutorial3` in the MarkUs repo and push the changes. Remember to click on the `tutorial3` on the MarkUs website, so it can add the `tutorial3` folder to your MarkUs repo, then `git pull` to add the `tutorial3` folder in your local copy of the repo.